

Technische Dokumentation für Spendly

LB3 Mobile Applikation realisieren

Darian Germann und Alex Platret

UBS APPLI A24 | Jörg Negwer | 1. Oktober 2026

Zweck der App

Spendly hilft Lernenden und Studierenden in der Schweiz, ihre täglichen Ausgaben in CHF zu erfassen und das verbleibende Monatsbudget zu verstehen. Aus Betrag, Kategorie und Datum entsteht eine Übersicht mit Restbudget und einer Orientierung pro verbleibendem Tag.

Stand des Endprodukts

Die App verbindet fünf Hauptscreens mit lokaler Speicherung, Formularprüfung und einer Zugangssperre. Diese Dokumentation beschreibt den aktuellen Aufbau, führt durch die Bedienung und vergleicht die Umsetzung mit dem LB1-Konzept. Die Testübersicht nennt nachgewiesene Ergebnisse und offene Grenzen getrennt.

Inhalt	Seiten
Aufgabe und Umfang	2
Aufbau und Navigation	3
Bedienung mit Screenshots	4 bis 6
Design und Ergonomie	7
Daten und Smartphonefunktion	8 bis 9
Konzept und Umsetzung	10
Codequalität und Prüfung	11 bis 12
Rückblick und Ausblick	13

Die Screenshots zeigen die tatsächlich laufende App. Die Abbildung auf Seite 7 mit der Bezeichnung LB1 ist ein ursprünglicher Entwurf.

Aufgabe und Umfang

Kleine Einkäufe sind einzeln überschaubar. Ohne gemeinsame Übersicht bleibt trotzdem unklar, wie viel vom verfügbaren Geld bis zum Monatsende übrig ist.

Zielgruppe und Nutzung

Spendly richtet sich an Lernende und Studierende von etwa 16 bis 25 Jahren. Eine typische Nutzung ist das Erfassen eines Einkaufs direkt an der Kasse oder unterwegs. Dafür sind kurze Beschriftungen, grosse Touch-Flächen und wenige Pflichtfelder vorgesehen. Ein Bankkonto wird nicht verbunden.

Kernaktivität	Ergebnis in Spendly
Budget prüfen	Monatsausgaben, Restbetrag und Tagesbudget sind sichtbar.
Ausgabe erfassen	Betrag, Kategorie, Beschreibung und Datum werden lokal gespeichert.
Ausgaben korrigieren	Suche und Filter führen zum Eintrag; Bearbeiten und Löschen sind erreichbar.
Zugang schützen	Biometrie oder ein eigener sechsstelliger Code geben die App frei.
Einstellungen verwalten	Budget, Darstellung und automatische Sperre bleiben gespeichert.

Bewusste Abgrenzung

Zum Umfang gehören manuelle Ausgaben in CHF und die lokale Nutzung auf dem Smartphone. Bankanbindung, automatischer Zahlungsimport, gemeinsame Konten, Cloud-Synchronisierung und Finanzberatung sind nicht Teil der App. Die Budgetzahl ist eine Rechenhilfe; sie bewertet keine finanzielle Entscheidung.

Erfolgskriterien

Ein gültiger Eintrag muss nach einem Neustart erhalten bleiben. Ungültige Beträge dürfen nicht gespeichert werden. Änderungen müssen Liste und Budget konsistent aktualisieren. Die Zugangssperre muss auch nach einem Neustart greifen. Das LB1-Ziel einer Erfassung in weniger als zehn Sekunden ist ein Nutzungsziel, das mit Menschen gemessen werden muss.

Die Erfassungszeit wurde bisher nicht als Nutzertest gemessen. Ein schneller automatisierter Durchlauf belegt dieses Ziel nicht.

Vorgehen im Projekt

Die Git-Historie dokumentiert am 23. September Setup, Theme und Navigation. Am 24. September folgten SQLite, Formularprüfung und Biometrie. Am 25. September wurden Sonderfälle, Tests und die Anleitung ergänzt. Die LB3-Finalisierung gleicht diesen Stand mit dem Konzept ab und vervollständigt konkrete Lücken.

Aufbau und Navigation

Spendly verwendet React Native mit Expo. React Navigation organisiert die Bildschirme; ein gemeinsamer Datenkontext stellt Ausgaben und Einstellungen bereit.

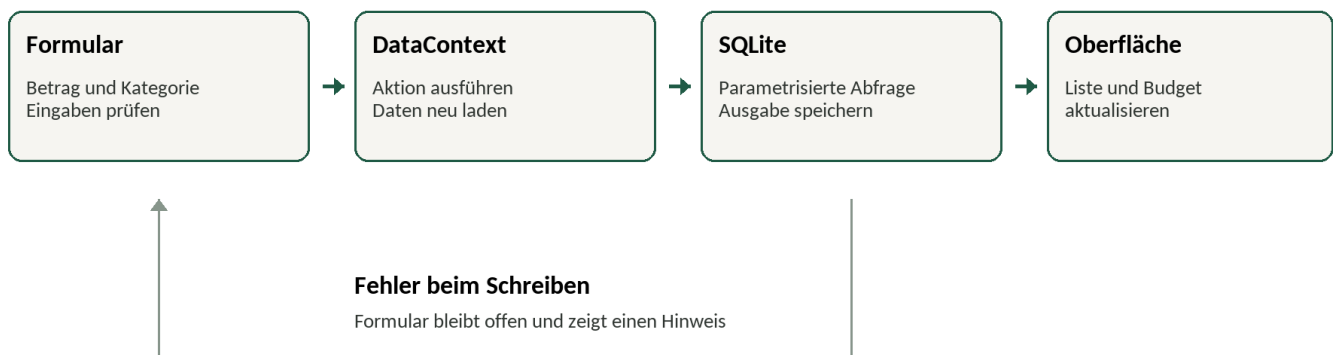
Bereich	Aufgabe
App.js	Lädt Schriften und verbindet Safe Area, Daten, Theme und Navigation.
screens	Fünf Ansichten für Entsperren, Übersicht, Formular, Verlauf und Einstellungen.
components	Wiederverwendbare Buttons, Felder, Zeilen, Dialoge und Rückmeldungen.
storage	Zugriff auf SQLite und SecureStore sowie gemeinsame Datenaktionen.
utils	Budgetrechnung, Eingabeprüfung, Formatierung, Filter und Sperrregeln.
theme	Farben, Schriften und Abstände für helle und dunkle Darstellung.

Navigation aus Sicht der Person

Beim Start erscheint die Sperre. Nach erfolgreicher Prüfung ersetzt die Tab-Navigation den Sperrbildschirm. Übersicht, Verlauf und Einstellungen bleiben als beschriftete Tabs erreichbar. Das Pluszeichen öffnet das Formular als Modal. Ein Tipp auf eine bestehende Ausgabe öffnet dasselbe Formular mit den gespeicherten Werten.

Speichern schliesst das Formular erst nach dem erfolgreichen Ablauf. Abbrechen oder Zurück führt zum Ausgangs-Tab. Ungespeicherte Änderungen werden vor dem Verwerfen bestätigt. Die manuelle oder automatische Sperre setzt die Navigation auf den Sperrbildschirm zurück.

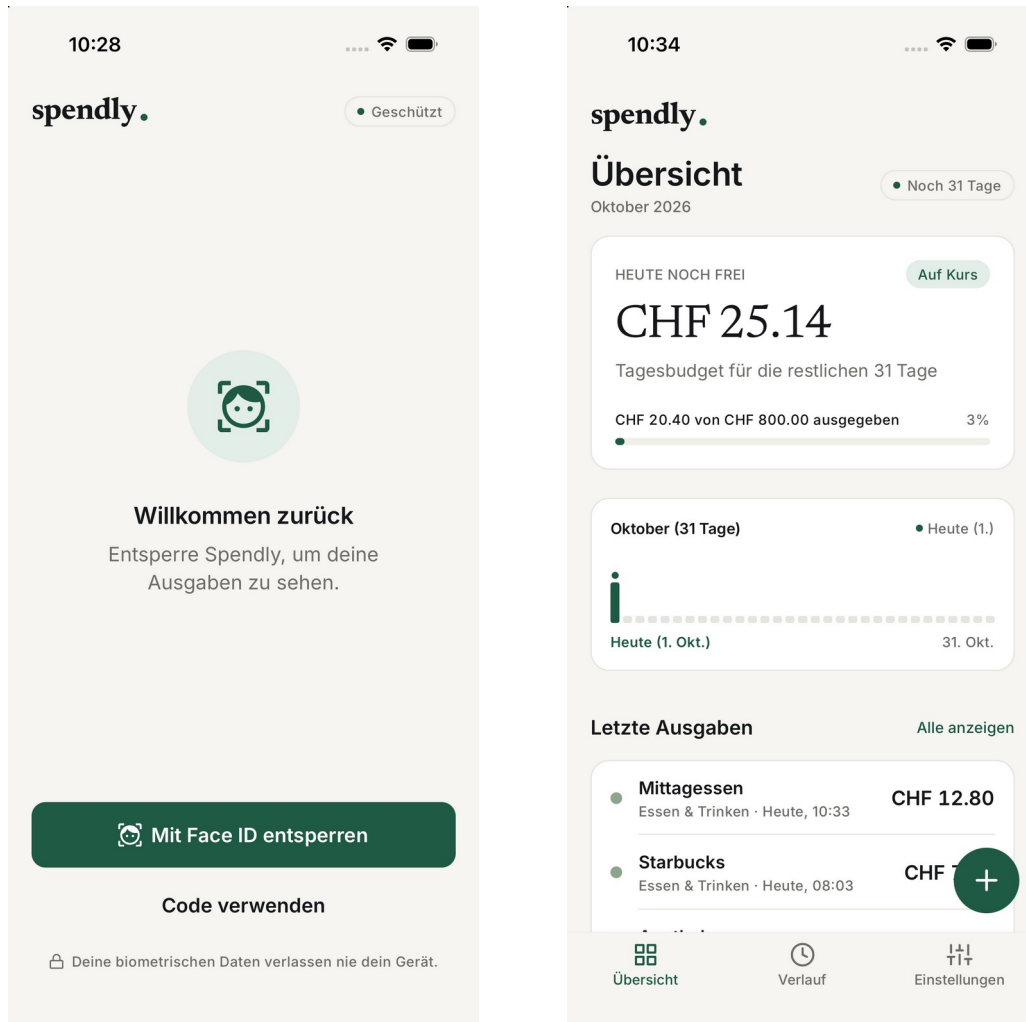
Weg einer neuen Ausgabe



Der Screen ruft eine Aktion aus useData auf. SQL bleibt in der Speicherschicht; Berechnung und Darstellung können dadurch getrennt geprüft werden.

App öffnen und Budget prüfen

Beim ersten Start wird ein eigener sechsstelliger Code eingerichtet. Danach steht die Übersicht im Mittelpunkt der täglichen Nutzung.



1 Zugang entsperren | 2 Monatsbudget überblicken

Vom Start zur Übersicht

Den Code zweimal eingeben und sicher merken. Bei späteren Starts mit dem Code oder verfügbarer, aktivierter Biometrie entsperren. Die Übersicht zeigt den aktuellen Monat, den ausgegebenen Betrag und das Restbudget. Der Rest wird auf die verbleibenden Tage einschliesslich heute verteilt.

Ohne gesetztes Budget führt «Budget festlegen» zu den Einstellungen. Bei einer Überschreitung erscheinen Fehlbetrag, Warnsymbol und Text. Die letzten Ausgaben sind direkt erreichbar; «Alle anzeigen» öffnet den Verlauf.

Ausgaben erfassen und ändern

Eine Ausgabe benötigt einen gültigen Betrag, eine Kategorie und ein Datum. Die Beschreibung ist optional. Das Formular unterstützt neue und bestehende Einträge.

The image displays two mobile app screens for recording a new expense. Both screens show the time 10:33 and the title 'Neue Ausgabe'.

Left Screen (Valid Entry):

- Section: BETRAG EINGEBEN
- Amount: CHF 14.80
- Feedback: Heute noch frei: CHF 25.56
- Section: Kategorie
- Selected: Essen & Trinken (checked)
- Other options: Mobilität, Freizeit, Shopping, Abos, Sonstiges
- Section: Beschreibung (optional)
- Description: Mittagessen
- Section: Datum
- Date: Heute, 1. Okt. 2026
- Button: Ausgabe speichern →

Right Screen (Invalid Entry):

- Section: BETRAG EINGEBEN
- Amount: CHF 0
- Error Message: ⚠ Betrag muss grösser als CHF 0.00 sein.
- Section: Kategorie
- Selected: Essen & Trinken (checked)
- Other options: Mobilität, Freizeit, Shopping, Abos, Sonstiges
- Section: Beschreibung (optional)
- Description: z. B. Migros, Zñüni
- Section: Datum
- Date: Heute, 1. Okt. 2026
- Button: Ausgabe speichern → (disabled)

3 Ausgabe erfassen | 4 Ungültige Eingabe erkennen

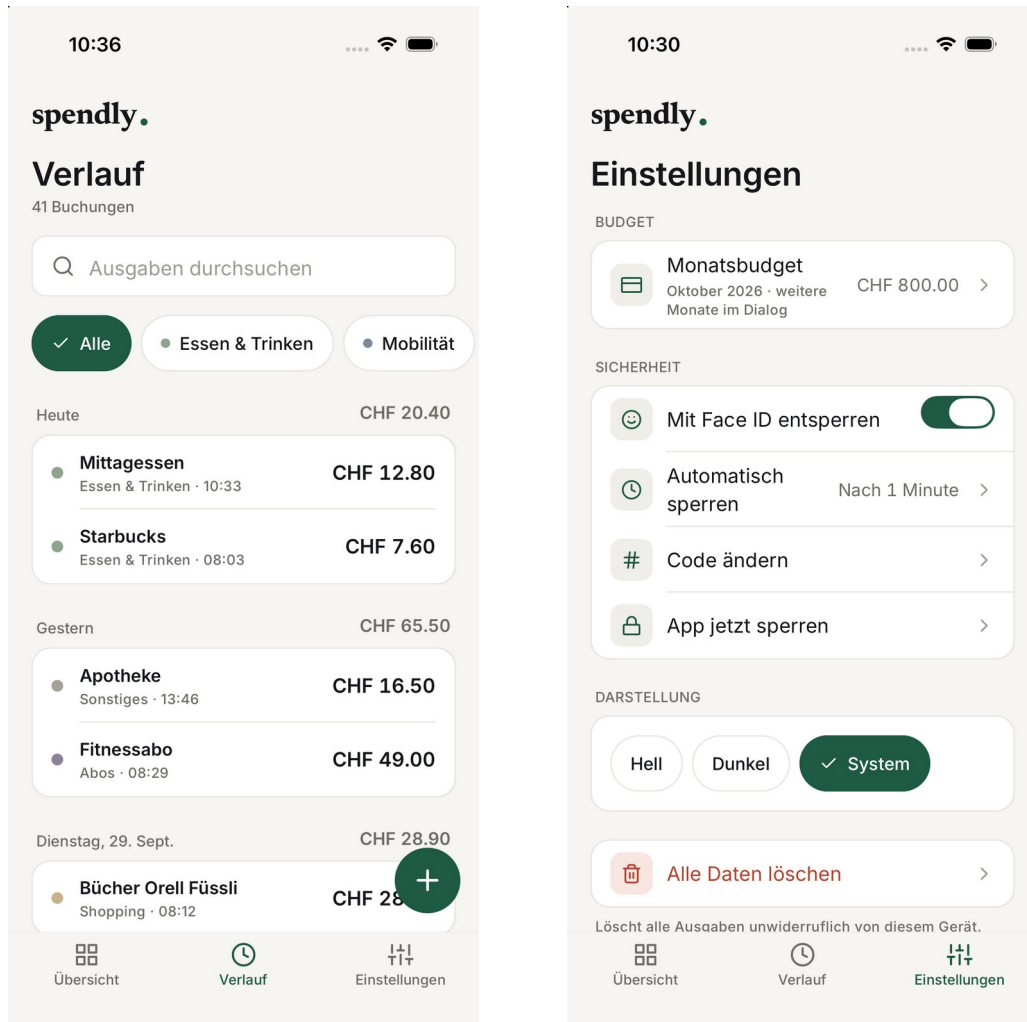
Auf das Pluszeichen tippen, den Betrag in CHF eingeben und eine der sechs Kategorien wählen. Punkt und Komma werden als Dezimaltrenner akzeptiert. Heute und die zuletzt verwendete Kategorie erleichtern wiederholte Eingaben. Eine Beschreibung wie «Migros» macht die Ausgabe im Verlauf leichter auffindbar.

«Ausgabe speichern» bleibt bei ungültigen Eingaben gesperrt. Eine verständliche Meldung nennt das betroffene Feld. Beim Bearbeiten lädt Spendly die ursprünglichen Werte und aktualisiert dieselbe Ausgabe. Beim Verlassen eines geänderten Formulars muss das Verwerfen bestätigt werden.

Bei einem Schreibfehler bleibt das Formular mit den Eingaben offen. Nach erfolgreichem Speichern werden Liste und Budget aktualisiert; eine kurze Rückmeldung bestätigt die Aktion.

Verlauf und Einstellungen

Im Verlauf werden Einträge gefunden und korrigiert. Die Einstellungen bündeln Monatsbudget, Zugang und Darstellung.



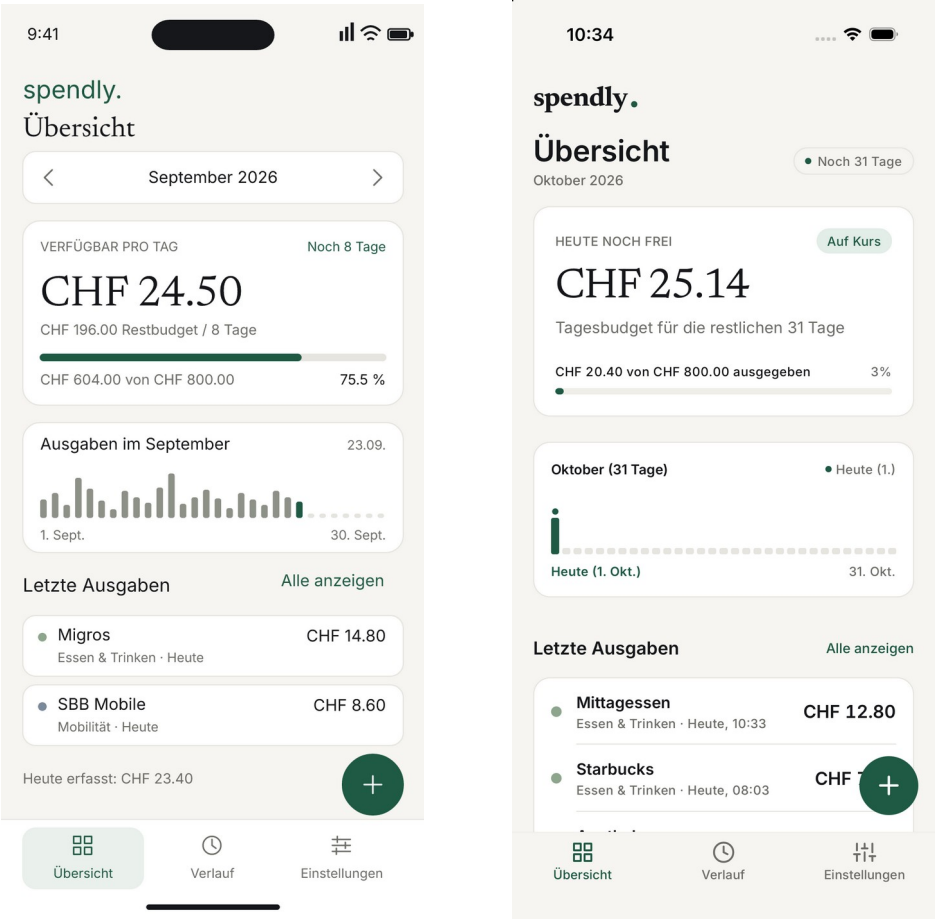
5 Ausgaben finden und korrigieren | 6 Budget und Zugang einstellen

Die Suche und der Kategorie-Filter grenzen die Liste gemeinsam ein. Die Gruppierung nach Datum mit Tagessumme erleichtert den Vergleich. Eine Zeile öffnet den Eintrag zum Bearbeiten. Löschen ist über die Swipe-Aktion und im Bearbeiten-Formular möglich; «Rückgängig» ist fünf Sekunden verfügbar.

In den Einstellungen öffnet «Monatsbudget» den Dialog für Monat, Betrag und Übernahme für künftige Monate. Dort lassen sich ältere Monatsbudgets ansehen und bearbeiten. Hell, Dunkel oder System bestimmen die Darstellung. «App jetzt sperren» schützt den Zugang sofort. «Alle Daten löschen» verlangt eine Bestätigung und führt zur Ersteinrichtung.

Design und Ergonomie

Die Umsetzung übernimmt die ruhige Hierarchie aus LB1. Grosse Budgetbeträge und eine feste Tab-Navigation machen die wichtigste Information und den nächsten Schritt sichtbar.



LB1 Entwurf der Übersicht | LB3 Tatsächlich laufende App

Entscheid	Begründung
Off White und Waldgrün	Ein heller Hintergrund und wenige Akzente lenken auf Beträge und Aktionen.
Newsreader und Inter	Serifenschrift hebt grosse Beträge hervor; Inter hält Felder und Listen gut lesbar.
Feedback mit Text	Fehler, Budgetüberschreitung und Löschen werden zusätzlich zur Farbe erklärt.
Beschriftete Tabs	Übersicht, Verlauf und Einstellungen bleiben nachvollziehbar erreichbar.
Dunkle Darstellung	Ergänzt den Mindestumfang von LB1 mit einem gespeicherten Erscheinungsbild.

Touch-Flächen sind für mindestens 48 logische Einheiten geplant. Safe Areas und scrollbare Inhalte berücksichtigen Gerätekanten und kleinere Displays. Ein vollständiger Nachweis mit grosser Systemschrift, VoiceOver und TalkBack erfordert eigene Gerätetests.

Datenhaltung und Budgetrechnung

Ausgaben und Einstellungen liegen lokal in SQLite. Der App-Code wird getrennt in SecureStore abgelegt. Es gibt keinen Spendly-Server und kein Benutzerkonto.

Speicher	Wichtige Werte	Zweck
expenses	id, amount_rappen, category, description, date, created_at, pending_delete_until	Einträge und befristetes Löschen
monthly_budgets	month, amount_rappen	Eigenes Budget je Kalendermonat
settings	Standardbudget, letzte Kategorie, Biometrie, Sperrzeit, Erscheinungsbild	Persönliche Vorgaben
SecureStore	Salt und Code-Hash; Fehlversuche und Sperrzeitpunkt	Getrennte Ablage für den Zugang
Nur im Arbeitsspeicher	Suchbegriff, Filter, offene Formulare und Entsperrt-Status	Vorübergehender Bedienzustand

Beträge als ganze Rappen

CHF 14.80 wird als 1480 gespeichert. So beruhen Summen auf ganzen Zahlen und nicht auf ungenauen Dezimaldarstellungen. Gebundene SQL-Parameter trennen Daten von SQL-Befehlen. Monats- und Tagessummen werden aus den relevanten Einträgen berechnet; zum Löschen vorgemerkte Einträge zählen nicht mit.

Nachvollziehbares Rechenbeispiel

Im LB1-Beispiel am 23. September beträgt das Monatsbudget CHF 800.00. Nach CHF 604.00 Ausgaben bleiben CHF 196.00. Vom 23. bis 30. September sind es acht Tage einschliesslich heute. Daraus ergeben sich CHF 24.50 pro verbleibendem Tag.

Spendly rundet das Tagesbudget auf ganze Rappen ab. Ein negativer Rest wird als Überschreitung angezeigt; die Tagesorientierung bleibt dann CHF 0.00. Bei älteren Monaten zeigt die App Monatsausgaben und Restbudget, weil dort kein verbleibender Tag mehr sinnvoll ist.

Monatswechsel und Rückgängig

Jeder Monat behält sein eigenes Budget. Eine neue Vorgabe gilt für noch nicht gespeicherte Monate; vorhandene Monatsbudgets bleiben erhalten. Beim Upgrade aus Version 1 wird das bisherige globale Budget auf vorhandene Ausgabemonate übertragen. Diese Werte sind eine Übernahme des alten Budgets, keine rekonstruierte Budgethistorie.

Löschen setzt eine Frist von fünf Sekunden. Rückgängig entfernt die Markierung; abgelaufene Markierungen werden bereinigt, auch nach einem Neustart. Die Schema-Version wird nach erfolgreicher Migration gespeichert.

Die SQLite-Datenbank ist in dieser Konfiguration nicht verschlüsselt. Die App-Sperre schützt den Zugang innerhalb der App. Es gibt keinen eigenen Export zur Sicherung und keine Cloud-Wiederherstellung.

Biometrie als Smartphonefunktion

Das Smartphone übernimmt die Erkennung von Gesicht oder Fingerabdruck. Spendly erhält nur das Ergebnis der Prüfung; die App liest keine biometrische Vorlage.

Erklärung für Laien

Die App fragt das Betriebssystem, ob eine passende Erkennung verfügbar und eingerichtet ist. Wenn die Person Biometrie aktiviert hat, öffnet Spendly den Systemdialog. Ein erfolgreiches Ergebnis gibt die App frei. Bei einem Abbruch bleiben die Inhalte gesperrt. Ein eigener App-Code ist als Alternative erreichbar.

Situation	Reaktion der App
Erkennung erfolgreich	Die geschützte Übersicht wird geöffnet.
Scan abgebrochen	Die App bleibt gesperrt; Abbruch zählt nicht als falscher Scan.
Drei fehlgeschlagene Scans	Die Eingabe des eigenen App-Codes wird angeboten.
Kein Sensor oder nicht eingerichtet	Ein Hinweis erklärt den Grund; der App-Code bleibt nutzbar.
Fünf falsche App-Codes	30 Sekunden Wartezeit; weitere Fehlversuche verlängern sie bis auf fünf Minuten.
Code vergessen	Nur aktivierte und verfügbare Biometrie erlaubt nach Prüfung einen neuen Code. Sonst bleibt ein bestätigter lokaler Neustart.

Technische Einbindung

expo-local-authentication prüft Hardware und Einrichtung und startet die Systemprüfung. Die Bezeichnung folgt dem verfügbaren Verfahren. Der Gerätecode-Fallback ist vom eigenen Spendly-Code getrennt. SecureStore hält den gesalzenen SHA-256-Prüfwert; der Code selbst wird nicht im Klartext gespeichert.

Die Umsetzung verwendet weiterhin SHA-256 mit zufälligem Salt. Das ist keine aufwendige Passwortableitung wie im LB1-Konzept vorgesehen. Die zusätzliche Sperrzeit begrenzt Versuche über die App, ersetzt jedoch keine stärkere Ableitung gegen einen ausserhalb der App geprüften Hash. Eine solche Umstellung bleibt ein dokumentierter Verbesserungspunkt.

Automatische Sperre und Nachweis

Die Voreinstellung sperrt nach einer Minute im Hintergrund. Sofort, fünf Minuten und nie sind weitere Optionen. Ein Neustart beginnt gesperrt; der Entsperrt-Status wird nicht gespeichert.

Der eigene iOS-Release-Build wurde auf dem iPhone-17-Pro-Simulator mit iOS 26.5 geprüft. Der native Face-ID-Systemdialog und «Matching Face» entsperren die App. Dies ist eine simulierte Erkennung. Für das physische iPhone wurde der signierte Release-Build erfolgreich erstellt und die Signatur geprüft. Installation und echter Gesichtsscan bleiben offen, weil Xcode das verbundene Gerät nicht erreichen konnte.

Konzept und heutige Umsetzung

LB1 bildet die Grundlage. Die Gegenüberstellung unterscheidet übernommene Funktionen, Ergänzungen und bewusst offen ausgewiesene Unterschiede.

LB1 Planung	Stand von LB3	Einordnung
Fünf Hauptscreens mit Tabs und Modal	Entsperren, Übersicht, Formular, Verlauf und Einstellungen vorhanden.	Übernommen
Lokale Ausgaben und CHF in Rappen	SQLite speichert Einträge; Summen beziehen sich auf das Ausgabedatum.	Übernommen
Budget pro Monat und Übernahme	Monatsbudgets, Standardbudget und historische Monate ergänzt.	Vervollständigt
Biometrie und eigener App-Code	Native Systemprüfung, Code-Einrichtung, Änderung und Wiederherstellung.	Umgesetzt
Gespeicherte Fehlversuche und Wartezeit	App-Code-Versuche und Sperrzeit überleben den Neustart.	Vervollständigt
Passwortableitung für den Code	Salt und SHA-256 bleiben bestehen; keine teure Passwortableitung.	Abweichung
Rückgängig für fünf Sekunden	Löschmarkierung mit Frist in der Datenbank; Bereinigung nach Ablauf.	Vervollständigt
Ungesicherte Änderungen bestätigen	Das Verlassen des geänderten Formulars verlangt eine Entscheidung.	Vervollständigt
Helle Darstellung als Mindestumfang	Hell, Dunkel und System sind wählbar und werden gespeichert.	Ergänzt
Einfache Ausgabe unter zehn Sekunden	Bedienung vorbereitet; noch kein gemessener Nutzertest.	Nicht belegt

Bedeutung der Anpassungen

Die Ergänzungen bei Monatsbudgets, Löschfrist und PIN-Versuchen schliessen konkrete Unterschiede zwischen Konzept und Implementierung. Sie verändern keine grundlegende Navigation. Der zusätzliche Dark Mode nutzt dasselbe Theme-System und denselben Funktionsumfang.

Die Code-Ableitung und noch nicht vollständig geprüfte Bedienbarkeit werden nicht als erledigt dargestellt. Diese Grenzen sind Teil der technischen Einordnung und des Ausblicks, damit Dokumentation und Produktstand übereinstimmen.

Codequalität und Eingabeprüfung

Die Codequalität entsteht aus klaren Zuständigkeiten, kleinen wiederverwendbaren Bausteinen und überprüfbaren Regeln. Kommentare erklären Entscheidungen und Abläufe.

Trennung der Aufgaben

Die Screens zeigen Daten und rufen Aktionen auf. DataContext koordiniert Speicherung und Aktualisierung. SQL steht in storage; reine Funktionen in utils lassen sich ohne Oberfläche testen. Wiederverwendbare Komponenten halten Darstellung und Bedienung konsistent. Die Theme-Dateien bündeln Farben, Schriften und Abstände.

Eingabe	Prüfregel	Rückmeldung
Betrag	Pflicht, grösser als null, höchstens zwei Nachkommastellen	Fehler direkt am Feld
Kategorie	Eine der sechs festen Kategorien	Auswahl verlangen
Beschreibung	Optional, maximal 40 Zeichen	Eingabe begrenzen
Datum	Gültiges Datum, nicht in der Zukunft	Datum korrigieren
Monatsbudget	Grösser als null, höchstens zwei Nachkommastellen	Speichern erst bei gültigem Wert
App-Code	Sechs Ziffern, Bestätigung bei Einrichtung	Übereinstimmung prüfen

Fehler und wiederholte Aktionen

Während einer Speicherung verhindert ein gesperrter Button doppelte Eingaben. Fehlgeschlagene Datenbankzugriffe werden als verständliche Meldung angezeigt. Beim Start bietet ein Ladefehler einen erneuten Versuch an. Destruktive Aktionen nennen ihre Folgen und können vor der Bestätigung abgebrochen werden.

Eingesetzte Bibliotheken

React Native und Expo bilden die Grundlage. React Navigation übernimmt Stack und Tabs. expo-sqlite speichert Daten; expo-secure-store und expo-crypto unterstützen die Codeprüfung. expo-local-authentication verbindet die Gerätesicherheit. Hinzu kommen expo-haptics für Rückmeldungen, der native DateTimePicker, Safe Area, Inter, Newsreader und Icons. Eine zusätzliche UI-Komponentenbibliothek wird nicht eingesetzt.

Expo SDK 57, React Native 0.86.3 und React 19.2.3 entsprechen package.json. Die festgehaltenen Abhängigkeiten stehen zusätzlich in package-lock.json.

Prüfung und bekannte Grenzen

Die folgenden Ergebnisse beziehen sich auf den dokumentierten Stand am 1. Oktober 2026. Automatische Prüfungen und beobachtetes Verhalten auf Geräten beantworten unterschiedliche Fragen.

Prüfung	Ergebnis und Aussage
Automatisierte Tests	91 von 91 Tests in neun Testdateien bestanden: Berechnungen, Eingaben, PIN-Sperre, SQLite, Biometrie und Wiederherstellungsdialog.
SQLite und Migration	Neun Tests mit echtem SQL über node:sqlite: Migration, Monatsbudgets, Transaktionen, Rollback und Löschrufen.
PIN und Biometrie	Regeln, Systemantworten und der Wiederherstellungsdialog bei aktivierter oder deaktivierter Biometrie sind getestet. Gerätesensoren bleiben simuliert.
ESLint und Diff-Prüfung	Ohne Fehler bestanden; geprüft werden Stilregeln und Formatierung.
iOS Bedienablauf	Im eigenen Release-Build aufgenommen: Erstellen, Bearbeiten, Suche, Löschen, Rückgängig und Neustart ohne Server. Monatsbudgets und Dark Mode geprüft.
Native Biometrie	Face-ID-Systemdialog und erfolgreiche simulierte Erkennung im iOS-Simulator. Signierter iPhone-Build erfolgreich; echter Scan auf dem physischen Gerät offen.
Android und Barrierefreiheit	Kein vollständiger neuer Android-, VoiceOver- oder TalkBack-Durchgang dokumentiert.
Erfassung unter zehn Sekunden	Noch nicht mit Personen der Zielgruppe gemessen.

Was zusätzlich geprüft werden muss

SQLite-Regressionen prüfen die SQL-Logik über node:sqlite. Der native Simulatorlauf ergänzt Bedienablauf und iOS-Modul. Echte Biometrie, Android, Screenreader und ein Zielgruppen-Nutzertest benötigen separate Nachweise.

Daten und Sicherheit

Es gibt keine eigene Sicherung oder Cloud-Wiederherstellung. Ein bestätigtes Löschen aller Daten ist endgültig. Die SQLite-Datenbank ist nicht verschlüsselt. Die Grenzen der Code-Ableitung sind auf Seite 9 beschrieben.

Testnachweise im Projekt

TESTKONZEPT.md nennt manuelle Fälle, Umgebung und Resultate. Erneut prüfen mit `npm test -- --runInBand` und `npm run lint`. Wegen node:sqlite benötigen die Tests Node.js ab 22.13; der dokumentierte Lauf nutzte 25.9.

Branch codex/lb3-finalisierung · Expo 57.0.26 · React Native 0.86.3 · native Release-Builds mit Xcode 27.0.

Rückblick und Ausblick

Die technische Arbeit zeigt vor allem, dass eine gut sichtbare Funktion auch ein klares Verhalten bei Fehlern, Neustarts und Monatswechseln benötigt.

Konkrete Erkenntnisse

Ein einzelner Budgetwert reicht für den aktuellen Monat, verliert aber die ursprüngliche Planung historischer Monate. Monatsbudgets machen diese Zuordnung ausdrücklich. Eine Löschaktion nur im Arbeitsspeicher ist einfach, doch ein Neustart beendet die Rückgängig-Möglichkeit. Eine gespeicherte Frist hält Daten und Bedienung konsistent.

Die Trennung von Oberfläche, Datenzugriff und reiner Berechnung erleichtert gezielte Änderungen. Gleichzeitig braucht Biometrie einen nativen Testweg. Ein optisch fertiger Sperrbildschirm und eine erfolgreich geprüfte Codeeingabe sind dafür keine ausreichenden Nachweise.

Ausblick: mögliche Erweiterungen

Nach den offenen Geräte- und Nutzertests könnten wir Spendly um einen CSV-Export und Statistiken nach Kategorien erweitern. Diese Funktionen sind Ideen für spätere Versionen und noch nicht umgesetzt. Welche Erweiterung hilft, soll Feedback der Zielgruppe zeigen. Die stärkere Code-Ableitung bleibt ein technischer Verbesserungspunkt.

App starten und präsentieren

Den Quellcode aus dem ZIP entpacken und im Projektordner `npm ci` ausführen. Mit `npx expo start` startet der Entwicklungsserver. Expo Go eignet sich für den Codeweg; Face ID auf iOS benötigt einen eigenen nativen Build. Ein Release-Build kann ohne laufenden Entwicklungsserver präsentiert werden. Die genaue Gerätevorbereitung steht in der README.

Ein eigener iOS-Release-Build wurde im Simulator installiert. Erfassung, Bearbeitung, Löschen, Rückgängig und vollständiger Neustart funktionierten mit gestopptem Entwicklungsserver. Die Abgabe enthält die Aufnahme dieses Durchlaufs. Der zusätzliche signierte Build für das physische iPhone ist vorhanden; Installation und echter Face-ID-Scan sind noch nicht nachgewiesen.

Grundlagen dieser Dokumentation

Spendly LB1, Konzept vom 24. September 2026; Anforderungen LB3, Abschnitte zu Präsentation, technischer Dokumentation und Bewertung; Prüfung 3 M335, Seite 2; Spendly-Quellcode mit README und TESTKONZEPT sowie die dokumentierten Prüfungen vom 1. Oktober 2026.

Repository: <https://github.com/germandarian/Spendly>

Expo Referenzen zur technischen Einbindung, abgerufen am 1. Oktober 2026:

LocalAuthentication: <https://docs.expo.dev/versions/latest/sdk/local-authentication/>

SQLite: <https://docs.expo.dev/versions/latest/sdk/sqlite/>

SecureStore: <https://docs.expo.dev/versions/latest/sdk/securestore/>